BLOCKFIT / PROJECT FIELD GUIDE

PRODUCT, ARCHITECTURE, IMPLEMENTATION & READINESS

Build BlockFit with the whole picture.

A practical guide to what the app is, what the repository contains, what works today, what is not ready yet, and how to move it toward a trustworthy consumer product.

Positioning: An AI-powered movement, fitness and rehabilitation companion.

Repository: `BlockFit_Enhanced`
Prepared: October 5, 2026
Scope: Current repository and documented product direction

How to use this guide

This is a comprehensive project orientation, not a line-by-line copy of the source code. It explains the important product decisions, runtime paths, feature boundaries, operational commands, known risks, and recommended work order. Treat code and tests as the final authority where older documents disagree.

> **Important:** The repository contains both implemented product functionality and planned/demo concepts. This guide labels the difference. Do not present a UI label, design mock, historical audit, or client-submitted score as proof that a capability is validated or production-ready.

1. Executive summary
2. Product and users
3. What works today
4. Architecture
5. Web and mobile code map
6. Pose and movement pipeline
7. Backend and data
8. Privacy, consent, safety
9. Run and test
10. Deploy and Android
11. Known gaps and risks
12. Recommended next steps
13. Glossary and key files

1. Executive summary

BlockFit is an existing full-stack MVP, not a blank-slate project. It already has a browser app, Express API, account flows, camera-based fitness analysis, workout persistence paths, progress views, and a Capacitor Android build path.

Keep

Working authentication, workout APIs, the MediaPipe wrapper, exercise analyzers, shared exercise catalog, consent-enforced clinician API, regression tests, and the existing web/Android architecture.

Be honest about

Movement quality is rules-based and not clinically validated. Only five listed exercises are camera-analyzable. Rehab is a labeled sample, and several profile values are static placeholders.

Product boundary

Support movement awareness, exercise feedback, and progress review. Do not diagnose, prescribe treatment, or claim to replace qualified physiotherapy.

Architecture goal

Keep pose analysis on-device, persist only necessary structured results, require explicit consent for clinician sharing, and build production operations around verified user flows.

---

2. Product and users

Product statement

BlockFit helps people observe exercise movement, receive immediate rule-based feedback, save workout summaries, and review progress. Its two primary modes are **Fitness** and **Physiotherapy / Rehabilitation**.

Initial market and trust

The initial target market is India. Privacy, clear consent, account deletion, accessible camera controls, honest claims, and region-appropriate legal review are product requirements, not polish. Privacy documentation mentions the DPDP Act 2023 and says the policy requires qualified legal review.

Users

- **Consumers / athletes:** start sessions, get basic exercise feedback, save results, inspect history, manage an account.
- **People following a rehab plan:** see a clearly non-prescriptive demo today; future prescribed plans need qualified clinical oversight and validated measurement.
- **Physiotherapists:** backend clinician connection and consent controls exist; a complete clinician-facing workflow is not yet a finished user product.

Product rules

- No public blockchain or token feature as a core product capability. Internal cryptographic integrity may support auditability.
- No claims of medical approval, clinical proof, injury diagnosis, or replacement of a physiotherapist.
- Do not imply that a high score is proof of correct biomechanics or a treatment outcome.
- Do not call deterministic exercise rules a trained BlockFit AI model.

3. What works today versus what is pending

Area	Current state	Safe description
Accounts	Register/login, password reset routes, session handling, profile retrieval/deletion are present.	Account functionality exists; production email and deployment configuration still matter.
Fitness camera session	MediaPipe landmarks feed five exercise-specific analyzers; reps and form feedback are shown.	Camera-based exercise feedback for listed supported exercises.
Exercise catalog	Squat, push-up, lunge, bicep curl, plank are marked available. Straight-leg raise, knee extension, shoulder raise are pending.	Five currently enabled movement analyzers; catalog count is not the same as implemented analysis.
Workout history	API save/list flow, dashboard and history surfaces exist.	Structured session summaries and user history.
Rehabilitation	Sample knee recovery plan is explicitly a demo, not a prescription. Rehab-specific analyzers are disabled.	Rehab concept and sample UI only; not a validated camera rehab program.
ROM / asymmetry	ROM route/utilities and architecture references exist, but the rehab UI says those measures are not yet measured.	Do not claim usable ROM or asymmetry tracking until end-to-end measurement and validation are verified.
Clinician access	Invite, claim, approval, revoke, connections, and server-side approval check for patient summary are present.	Consent-enforcement foundation exists; verify UI, authorization and lifecycle end to end.
Public verification	Verification endpoints and a public session summary page exist.	A shareable session summary; not independent validation of movement accuracy.
Android	Capacitor wrapper and Gradle project exist.	Build path exists; needs configured public API and physical-device validation.
Profile metrics/settings	The current profile design includes hard-coded score, streak, goal, level, and account setting labels.	Visual placeholders only until driven by real account/session data; do not represent them as actual user state.

The eight-entry catalog and demo schedule live in `shared/exerciseCatalog.mjs`. The sample schedule includes `supported-squat` and `heel-raise`, which are not among the eight catalog entries; this

reinforces that it is illustrative, not executable prescription.

---

4. Architecture at a glance

```
Browser / Capacitor WebView
  UI routes, auth session, camera permission
       |
       +-- Camera frames --> MediaPipe Pose Landmarker (on device)
       |                        --> confidence / landmark checks
       |                        --> exercise analyzer rules
       |                        --> reps, feedback, derived summary
       |
       +-- HTTPS / same-origin /api --> Express API
                                          --> auth + authorization
                                          --> workout / user / clinician / ROM /
verification routes
                                          --> PostgreSQL in production
                                          --> in-memory store for development/tests
```

Frontend and API are separated by an HTTP API boundary. The Android app packages the shared web UI using Capacitor and talks to the public API. Camera video is not part of a normal workout record; the architecture intends to save derived movement summaries instead.

Top-level folders

- `web/` browser SPA, active pose/exercise pipeline, and Vite build.
- `backend/` Express API, environment setup, persistence adapter, SQL schema and tests.
- `mobile/` Capacitor config and native Android Gradle project.
- `shared/` exercise catalog and shared domain definitions.
- `benchmark/` movement evaluation protocol and test harness.
- `docs/` architecture, safety, privacy, product and release guidance, including the repository reference map.

---

5. Web app and mobile code map

- `web/src/main.js` boots the browser application.

- `web/src/app.js` owns hash routing, authenticated route gates, shell/navigation, logout, and page mount/unmount behavior.
- `web/src/pages/landing.js`, `login.js`, `register.js` cover public entry and account flows.
- `web/src/pages/dashboard.js`, `workout.js`, `history.js`, `rehabilitation.js`, `profile.js` cover core app surfaces.
- `web/src/pages/verify.js` displays the public session summary.
- `web/src/pose/` wraps detection, movement confidence and geometry; `web/src/exercises/` holds exercise-specific analyzers; `web/src/ui/overlay.js` draws the skeleton overlay.
- `web/src/api.js`, `auth.js`, `analytics.js`, `coach.js` coordinate API access, session details, events and post-session feedback.
- `web/src/style.css` contains shared tokens, layout, responsive styling, animation and route-specific presentation.
- `mobile/build-apk.mjs` builds web assets, syncs Capacitor and assembles the Android APK.

Recent profile styling has hard-coded sample metric values and setting labels. Treat them as temporary display content, not backed settings. User-controlled text should be HTML-escaped or inserted using safe text APIs.

---

6. Pose and movement analysis

Current session sequence

1. User selects an enabled exercise and grants camera permission.
2. The app requests a fresh workout nonce from the authenticated API.
3. The MediaPipe pose landmarker loads and camera frames are read locally.
4. Pose confidence checks can request repositioning when landmarks are not reliable.
5. The selected analyzer updates movement phase, repetition count, live issues and completed-rep scores.
6. The overlay and coaching UI show movement cues, counts and a summary.
7. On finish, the client posts a structured summary to the API using the nonce; it may register a public verification summary separately.

What the score means

It is an output of current deterministic rules and camera landmark estimates. It can vary with camera placement, visibility, lighting, occlusion, device performance, body proportions and movement style. It is not a diagnosis, validated clinical measurement, or guarantee that an exercise is safe.

Important trust limitation

> The server validates workout fields and consumes a one-time nonce, but the submitted reps, scores, confidence and verification status originate in the client. A nonce helps prevent replay; it does not prove that the client's camera analysis was accurate or untampered. Treat public verification as a summary of submitted data, not clinical or cryptographic proof of exercise quality.

ML status

MediaPipe is the pose perception dependency. Exercise phase/repetition/form logic is rule-based. Future temporal models (such as TCN, GRU/LSTM or Transformer) are architecture possibilities only; no trained BlockFit model should be claimed.

---

7. Backend, API and data

Backend modules

- `backend/src/app.js`: Express composition, Helmet, CORS allow-list, JSON parsing, logging, API rate limit, health endpoint, route mounts, optional static web serving.
- `backend/src/server.js`: server startup and port binding.
- `backend/src/config/env.js`: dotenv loading, CORS origins, secrets, database/mail config and production secret checks.
- `backend/src/lib/auth.js`, `middleware/auth.js`: password/token helpers and protected route authentication.
- `backend/src/lib/dataStore.js`, `lib/store.js`: persistence interfaces and runtime storage selection.
- `backend/src/db/schema.sql`: relational schema baseline.
- `backend/src/routes/`: auth, users, workouts, ROM, verification, analytics and clinicians.

Route groups

Core endpoints are mounted below `/api`: `/auth`, `/users`, `/workouts`, `/rom`, `/verification`, `/analytics`, and `/clinicians`. Workout routes include authenticated nonce issuance, workout listing and workout creation. Clinician summaries require server-side approved access.

Persistence modes

- With `DATABASE_URL`, production is intended to use PostgreSQL.
- Without it, development/test mode uses in-memory storage; data disappears when the server restarts.

- Test persistence behavior deliberately. Do not confuse a passing in-memory test with proof that the deployed database, migrations, backups, and restore path are production ready.

Configuration

- `JWT_SECRET` and `INTEGRITY_HMAC_SECRET` are required in production; the code fails startup if absent.
- `DATABASE_URL` points at PostgreSQL. `PORT`, `NODE_ENV`, and `CLIENT_ORIGIN` configure runtime and CORS.
- Password-reset email needs SMTP host/port/security/user/password/from plus `CLIENT_APP_URL`.
- `CLINICIAN_ADMIN_SECRET` is an optional configured secret for admin-related clinician operations; keep all secrets server-side.

---

8. Privacy, consent and safety

Existing safeguards

- JWT-protected API routes and password hashing.
- Helmet, CORS allow-list, request rate limiting, input validation, and a 1 MB JSON request limit.
- Patient-controlled clinician invite/approval/revoke workflow with server-side access authorization.
- Account deletion and verification revocation flows.
- Internal SHA-256/HMAC integrity helpers; no public blockchain feature required.

Privacy promises require code verification

Current architecture and privacy documents say raw camera video is not uploaded/stored by default, with derived summaries persisted. The privacy policy also describes optional quantized pose-landmark storage and retention. Confirm the actual enabled routes, consent UI, storage schema, expiry job, deletion behavior, and deployment configuration before making a public data-handling promise.

Safety boundary

- Users need clear camera permissions, camera placement guidance, and low-confidence/reposition states.
- Stop, pause and discomfort pathways matter especially in rehabilitation.
- Never automatically change clinician-assigned treatment targets without an approved clinical process.
- Rehab plans and measurement claims need qualified clinical review and validation.
- Privacy policy still identifies a placeholder support contact and requires counsel review before public launch.

Read `docs/SAFETY.md`, `docs/PRIVACY_POLICY.md`, and `docs/PHYSIOTHERAPY_MODE.md` together. If their promises differ from code, resolve the mismatch before release.

---

9. Local setup, commands and test strategy

Requirements

Use a supported Node.js version (README specifies Node 20+ for Android; use a current supported LTS for development), npm, and a browser with camera access. Android builds additionally need JDK 21 and Android SDK tooling.

Install dependencies

```
npm ci --prefix backend
npm ci --prefix web
```

Run in development

Open two terminals at the repository root:

```
npm run dev:api
npm run dev:web
```

Visit `http://localhost:5173`. Vite proxies `/api` to the local backend on port 4000. On Windows, `start.bat` launches both services.

Run tests and build

```
node --test backend/test/*.test.js
npm run benchmark:test
npm run build:web
```

The backend tests include API smoke, movement assessment, persistence, privacy, verification and workout flows. Benchmark evaluation files live in `benchmark/`; review `benchmark/PROTOCOL.md` for what the benchmark does and does not establish.

Verification discipline

- Run the narrow test for a changed feature first, then the relevant suite and build.
- Check authenticated, unauthenticated, empty, error, deletion and revoke states, not only happy paths.

- For camera work, test real devices/camera angles and measure false counts, missed reps, confidence failures and score variance.
- For privacy changes, validate the backend authorization boundary and data lifecycle, not only frontend visibility.

---

10. Deployment and Android APK

Web/API deployment

`render.yaml` configures a Render Blueprint with the web app, API and PostgreSQL service. Deploying requires the owner's Render account; repository configuration alone does not publish the app. Production must set strong JWT and integrity secrets, database URL, allowed client origin, and any email settings needed by enabled auth flows.

The project can serve the Vite production build and API from one origin: `npm run build`, then `npm start`, then open `http://localhost:4000`. For local persistence, configure PostgreSQL; without it, development data is in memory.

Android

1. Deploy the API and obtain its public HTTPS URL.
2. Copy `mobile/api-config.example.json` to `mobile/api-config.json` and set the API URL ending in `/api`.
3. Install dependencies: `npm ci --prefix mobile`.
4. Build: `npm run apk:build`.
5. Test camera permission, API connectivity, lifecycle/background behavior, performance, network failures and account flows on physical Android devices.

APK output is documented as `mobile/android/app/build/outputs/apk/debug/app-debug.apk`. The public API address is embedded at build time; rebuild if it changes. Do not commit private API credentials or production secrets.

---

11. Known gaps and risks to manage

1. **Profile data is currently illustrative:** score, streak, goal, level and several account settings are hard-coded in the profile page. Replace with real fields or transparent unavailable states.
2. **Client-supplied workout metrics:** nonce consumption is not an independent validation mechanism. Confidence and verification status must not be treated as proof.

3. **Rehabilitation readiness:** three rehab analyzers are pending, the sample plan is non-prescriptive, and measured ROM/asymmetry are not yet shown as available.
4. **Clinical and legal review:** safety claims, rehab content, privacy and India-specific launch compliance need qualified review.
5. **Camera/model delivery:** pose model assets are loaded via a CDN according to the architecture/audit notes. Confirm offline behavior, caching, availability, licensing, and Android reliability.
6. **Production operations:** validate database migrations, backups, restore, monitoring, incident response, email delivery, secret rotation, and deletion workflows.
7. **Public verification semantics:** explain exactly what a shared record proves; it should not imply that a server observed or clinically validated the movement.
8. **Documentation drift:** the V2 audit is explicitly historical. Reconcile old blockchain/prototype language against current docs and actual code.
9. **Profile output encoding:** user-provided name/email should be safely rendered in HTML. Favor text APIs or escaping at every dynamic insertion point.

> **Release rule:** A polished interface is not evidence that a feature is real. Every consumer-visible metric, privacy promise, clinical claim and status label needs a data source, tested behavior and owner.

---

12. Recommended implementation order

1. **Truthful UI and data:** remove/replace static profile metrics; audit every screen for demo claims, disabled actions, empty states and accessibility.
2. **Movement quality:** build repeatable benchmark data and confidence handling for the five supported fitness movements; document camera setup and known failure modes.
3. **Security and integrity model:** define the meaning of nonce, client scores, server validation, verification records and audit events. Avoid calling client-generated summaries tamper-proof.
4. **Production readiness:** exercise PostgreSQL-backed flows, migration and backup/restore, account deletion, email reset, monitoring and incident procedures.
5. **Rehabilitation vertical:** work with qualified physiotherapy reviewers to define one scoped use case, safety/stop behavior, valid camera view, measurement reliability and consent before enabling rehab analyzers.
6. **Clinician workflow:** build a complete, least-privilege UI on top of the existing approval/revoke API and test every access transition.

7. **Android quality:** test supported devices, camera permission denial, offline/model load failures, app backgrounding, API timeouts and upgrades.
8. **Release and support:** finalize India-specific legal/privacy review, public support contact, retention/deletion operations, accessibility checks and beta incident process.

The current delivery plan is in `docs/PHASE_PLAN.md`. Change order only when user research or test evidence supports it.

---

13. Glossary and key files

Term	Meaning here
Pose landmark	Estimated body joint/keypoint location returned by pose perception, often with visibility/confidence information.
Analyzer	Exercise-specific deterministic logic that interprets landmarks over time to estimate phase, reps and form cues.
Workout nonce	Short-lived, single-use server-issued value checked when submitting a workout. It is not proof of movement accuracy.
ROM	Range of motion; a measurement that requires camera/geometry validation and must not be presented as clinical truth without evidence.
Consent-based access	Patient explicitly approves a clinician connection; server authorization checks access rather than trusting UI state.
Capacitor	Packaging layer that runs the shared web experience inside native Android shell capabilities.

- Current architecture: `docs/ARCHITECTURE.md`
- AI design/status: `docs/AI_ARCHITECTURE.md`
- Product roadmap: `docs/PHASE_PLAN.md`
- Safety: `docs/SAFETY.md`
- Privacy: `docs/PRIVACY_POLICY.md`
- Rehabilitation boundary: `docs/PHYSIOTHERAPY_MODE.md`
- Movement benchmark: `benchmark/PROTOCOL.md`
- Developer setup/deploy: `README.md`

> **Keep this guide current:** update feature status and commands when routes, analyzers, storage behavior or deployment configuration change. Recheck all truthfulness and privacy statements against code before a public release.
